

IA-32 and x86-64 Assembly

Christoffer Åleskog

Department of Computer Science
Blekinge Institute of Technology
Karlskrona, Sweden



Outline

- Overview
 - Lab, Supervision, and Assignment
- Motivation
 - Why use Assembly?
 - IA-32 & x86-64 History
 - IA-32 & x86-64 Architecture
- Assembly Language Instruction
 - Compiling and Writing Assembly Language
 - Handling of Data (variables & constants)
 - Instruction Format, Labels, and Registers
- Writing IA-32 / x86-64 Programs
 - Arithmetic / Logic Instructions
 - Program Flow (branches)
 - Referencing Memory
 - Interfacing with the Operating System

Outline

- Overview
 - Lab, Supervision, and Assignment
- Motivation
 - Why use Assembly?
 - IA-32 & x86-64 History
 - IA-32 & x86-64 Architecture
- Assembly Language Instruction
 - Compiling and Writing Assembly Language
 - Handling of Data (variables & constants)
 - Instruction Format, Labels, and Registers
- Writing IA-32 / x86-64 Programs
 - Arithmetic / Logic Instructions
 - Program Flow (branches)
 - Referencing Memory
 - Interfacing with the Operating System

Overview | Lab, Supervision, and Assignment

- Same structure as previous two modules
- Lab 14/10-2024, 15:00-18:00
 - Online Video Instruction
 - Room: Datorsal Minervasalen

Overview | Lab, Supervision, and Assignment

- Same structure as previous two modules
- Lab 14/10-2024, 15:00-18:00
 - Online Video Instruction
 - Room: Datorsal Minervasalen
- Assignment
 - Available after Lab
 - Grade A-F, 2.5 credit (*DV1457*) or 1 credit (*DV1578*)

Overview | Lab, Supervision, and Assignment

- Same structure as previous two modules
- Lab 14/10-2024, 15:00-18:00
 - Online Video Instruction
 - Room: Datorsal Minervasalen
- Assignment
 - Available after Lab
 - Grade A-F, 2.5 credit (*DV1457*) or 1 credit (*DV1578*)
- Supervision
 - Book 15-min time slots
 - Time slots will be posted on Monday

Outline

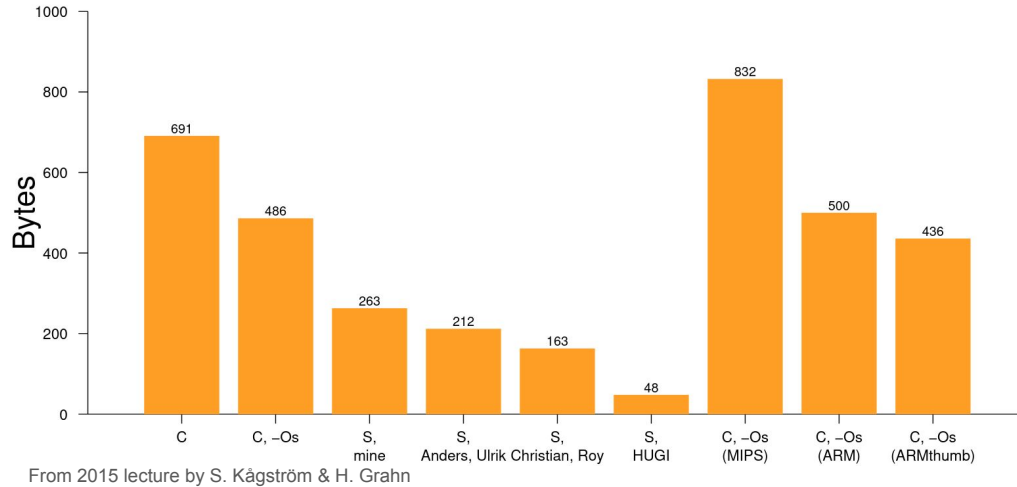
- Overview
 - Lab, Supervision, and Assignment
- Motivation
 - Why use Assembly?
 - IA-32 & x86-64 History
 - IA-32 & x86-64 Architecture
- Assembly Language Instruction
 - Compiling and Writing Assembly Language
 - Handling of Data (variables & constants)
 - Instruction Format, Labels, and Registers
- Writing IA-32 / x86-64 Programs
 - Arithmetic / Logic Instructions
 - Program Flow (branches)
 - Referencing Memory
 - Interfacing with the Operating System

Motivation | Why use Assembly?

- Speed?
 - Rarely. It is hard to optimize better than the compiler.
 - Sometimes: Writing vectorised code for DSPs
 - Skilled assembly programmers can also (sometimes) beat the compiler, but you must know the machine internals in order to manage that
- Operating systems?
 - Not directly, most OS code is not assembly (e.g., [C11 for Linux Kernel](#)). However, low-level hardware specific routines are necessary to write in assembly.

Motivation | Why use Assembly?

- Size?



- You can reduce the size by assembly implementations.
- But as you can see, the C-version is fairly small as well.
- This is seldom a real problem, except for small embedded devices.

Motivation | Why use Assembly?

- Know your machine?
 - For writing an OS? Know your machine
 - Exploiting security holes? Know your machine
 - Optimizing performance? Know your machine
 - Etc. etc.
- To know your machine: **Learn Assembly!**
- To learn assembly? Know your machine...

Motivation | IA-32 & x86-64 History I

- ISA (Instruction Set Architecture)
 - Interface between hardware and software.
 - CISC (Complex Instruction Set Computer) architecture
 - RISC (Reduced Instruction Set Computer) architecture
- What is IA-32?
 - IA-32 (Intel Architecture / 32 bits), CISC architecture.
 - x86, Intel32
 - Backwards compatible:

“One of the most important achievements of the IA-32 architecture is that the object code programs created for these processors starting in 1978 still execute on the latest processors in the IA-32 architecture family.” — [Intel Developers Manual](#)
 - IA-32 internally works RISC-like (microcode)
- AMD has implemented 64-bit extensions to IA-32
 - AMD64, Intel64, x86-64, x64

Motivation | IA-32 & x86-64 History II

- The Intel-architecture starts with the 8086 from the 1970s
- The 8086 is a 16-bit architecture (x86-16, data bus)
- 16-bit registers (ax, bx, cx, ...)
- Memory addresses are 16-bit (i.e. possible to address 64KB)
- “640 KB should be enough for anyone!” Why not 64KB?

Motivation | IA-32 & x86-64 History II

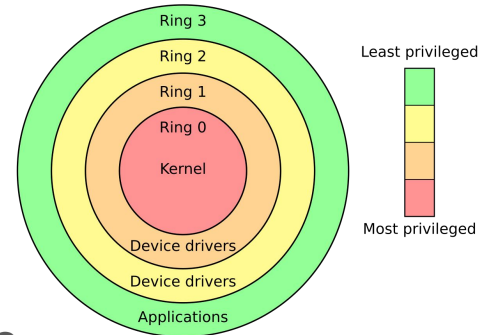
- The Intel-architecture starts with the 8086 from the 1970s
- The 8086 is a 16-bit architecture (x86-16, data bus)
- 16-bit registers (ax, bx, cx, ...)
- Memory addresses are 16-bit (i.e. possible to address 64KB)
- “640 KB should be enough for anyone!” Why not 64KB?
 - Segmentation: Register containing a base, addresses offsets to that base
 - Segment registers 16-bit wide
 - Different segments for code, data and stack (cs, ds, ss).
 - Real address (“effective address”): $\text{segment} * 16 + \text{address}$
 - $64\text{KB} * 16 = 1\text{MB}$ (640KB is an MS-DOS limit)
- We will use the 80386 and up (IA-32, later x86-64)

Motivation | IA-32 & x86-64 History III

- From 80386 and onwards, Intel CPUs use a 32-bit architecture
- 386: 4GB address space, protection etc.
- 486: Caching, better FPU, instruction pipelining
- Pentium: More pipelining, branch prediction, faster data bus
- P6 family (PPro, P-II, P-III): Redesign, superscalar, MMX/SSE, register renaming, larger caches, etc.
- P4: Redesign (NetBurst), Tracebuffer, hyperthreading
- x86-64/EM64T: 64-bit extension of IA-32, 8 extra registers, extended old registers, misc changes
- Core family (i3, i5, i7): Multicore, builtin graphics, turbo boost, power control, virtualization
- Itanium: Complete redesign, but emulates the old IA-32

Motivation | IA-32 & x86-64 Architecture I

- Four privilege levels: 0 (supervisor) ... 3 (user).
- Only 0 and 3 used in practice.
- In privilege level 0, all instructions are available (kernel mode).
 - e.g., setting the page directory, accessing control registers etc.
- Standard paging is available, 4KB and 4MB pages.
- Device communication (in/out) instructions can be (but is usually not) available in user-mode.
- Software interrupts (the int instruction) is generally not available in user-mode, but can be used for syscalls.



By Hertzprung at [English Wikipedia](https://en.wikipedia.org/wiki/IA-32_architecture#Privilege_levels), CC BY-SA 3.0

Motivation | IA-32 & x86-64 Architecture II

- There are a number of segment registers, *cs*, *ds*, *ss* etc.
- The segment registers specify an index into a table.
- Each table (the GDT or LDT) entry contains¹:
 - A base address (32 bits)
 - Base address is ignored in 64-bit mode
 - A segment limit (20 bits, 1B/4KB granularity)
 - Segment limit is ignored in 64-bit mode
 - Privilege level (0..3)
 - Misc information.
- It is not possible to change to a segment with a higher privilege level than the current one.

¹: See Volume 3A, Section 3.4.5: Segment Descriptors and Figure 3-8: Segment Descriptor of the [Intel Software Developer Manual](#)

Outline

- Overview
 - Lab, Supervision, and Assignment
- Motivation
 - Why use Assembly?
 - IA-32 & x86-64 History
 - IA-32 & x86-64 Architecture
- **Assembly Language Instruction**
 - Compiling and Writing Assembly Language
 - Handling of Data (variables & constants)
 - Instruction Format, Labels, and Registers
- Writing IA-32 / x86-64 Programs
 - Arithmetic / Logic Instructions
 - Program Flow (branches)
 - Referencing Memory
 - Interfacing with the Operating System

ASM Inst. | Compiling and Writing Assembly Language

- Assembly source are put in *.asm*-files (also could be *.s* or *.S*)
- The *.asm*-files are compiled by the assembler into object files (*.o*)
- The object files are linked together by the linker into an executable file
- Note that this is exactly the same process as for C/C++ source!
- UNIX systems use the ELF object/exec format
 - Data section (*.data*): Initialised data (global variables)
 - BSS section (*.bss*): Uninitialised data (globals, zero)
 - Text section (*.text*): Code
 - Misc other sections
- We will use the *AT&T*-style instead of Intel-style assembly
 - *AT&T*-style is from Bell Labs, and GCC defaults to it.

ASM Inst. | Compiling and Writing Assembly Language

- Coding guidelines
 - Program code is found in the `.text`-segment
 - Code for `GCC (gcc)` or `GNU (as)` looks as below code
 - This is `AT&T`-style assembly which you can recognize by
 - %-prefixed registers,
 - \$-prefixed constants,
 - Operation *source*, *dest*
 - Regarding comments: Stick to one style...
 - Depending on the architecture, different comment styles might work and other not, e.g., only “# ...” and “/* ... */” work on my machine (x86-64 running Fedora)
- Look at “`as`” documentation¹ for info on supported directives (e.g., `.section`, `.global`, `.asciz`)

```
1 .text
2     movl    $5, %rax          # rax = 5
3     subl    %rax, %rbx        // rbx = rbx - rax
4     mull    %rbx              /* rax = rax * rbx */
```

¹: “`as`” documentation is linked in the [GNU's binutils webpage](https://sourceware.org/binutils/docs/as.html), the documentation itself is at <https://sourceware.org/binutils/docs/as.html>

ASM Inst. | Compiling and Writing Assembly Language

- Coding guidelines
 - Program code is found in the `.text`-segment
 - Code for `GCC (gcc)` or `GNU (as)` looks as below code
 - This is `AT&T`-style assembly which you can recognize by
 - %-prefixed registers,
 - \$-prefixed constants,
 - Operation *source, dest*
 - Regarding comments: Stick to one style...
 - Depending on the architecture, different comment styles might work and other not, e.g., only “# ...” and “/* ... */” work on my machine (x86-64 running Fedora)
- Look at “`as`” documentation¹ for info on supported directives (e.g., `.section`, `.global`, `.asciz`)

```
1 .text
2     movl    $5, %rax          # rax = 5
3     subl    %rax, %rbx        // rbx = rbx - rax
4     mull    %rbx              /* rax = rax * rbx */
```

¹: “`as`” documentation is linked in the [GNU's binutils webpage](https://sourceware.org/binutils/docs/as.html), the documentation itself is at <https://sourceware.org/binutils/docs/as.html>

ASM Inst. | Compiling and Writing Assembly Language

- Coding guidelines
 - Program code is found in the `.text`-segment
 - Code for `GCC (gcc)` or `GNU (as)` looks as below code
 - This is `AT&T`-style assembly which you can recognize by
 - %-prefixed registers,
 - \$-prefixed constants,
 - Operation *source*, *dest*
 - Regarding comments: Stick to one style...
 - Depending on the architecture, different comment styles might work and other not, e.g., only “# ...” and “/* ... */” worked on my machine (x86-64 running Fedora)
- Look at “`as`” documentation¹ for info on supported directives (e.g., `.section`, `.global`, `.asciz`)

```
1 .text
2     movl    $5, %rax
3     subl    %rax, %rbx
4     mull    %rbx
```

```
#   rax = 5
//  rbx = rbx - rax
/*  rax = rax * rbx */
```

¹: “`as`” documentation is linked in the [GNU's binutils webpage](https://sourceware.org/binutils/docs/as.html), the documentation itself is at <https://sourceware.org/binutils/docs/as.html>

ASM Inst. | Handling Data

- Data can be in either the **.bss (uninitialised)** or the **.data** (initialised) section
 - Data in **.bss** will always be **zero** at program startup
- Dynamic memory must be allocated via the system (malloc/sbrk)
 - Seldomly used in assembly

```
1 .bss
2     NR_BSS: .long 0           // Will be 0 at program startup
3     MORE: .space 128         // 128 bytes of zeroes
4
5 .data
6     NR_DATA: .long 0x123      // Hex value
7     .asciz "hej"             // string (i.e. bytes 0x6865600)
8
9 .section .text
10    movl NR_DATA, %eax        // NR DATA into %eax
11    movl %eax, NR_BSS        // Copy from NR_DATA to NR_BSS
```

ASM Inst. | Handling Data

- Data can be in either the `.bss` (uninitialised) or the `.data` (initialised) section
 - Data in `.bss` will always be **zero** at program startup
- Dynamic memory must be allocated via the system (malloc/sbrk)
 - Seldomly used in assembly

```
1 .bss
2     NR_BSS: .long 0      // Will be 0 at program startup
3     MORE: .space 128    // 128 bytes of zeroes
4
5 .data
6     NR_DATA: .long 0x123 // Hex value
7     .asciz "hej"        // string (i.e. bytes 0x6865600)
8
9 .section .text
10    movl NR_DATA, %eax    // NR DATA into %eax
11    movl %eax, NR_BSS    // Copy from NR_DATA to NR_BSS
```

ASM Inst. | Handling Data

- Data can be in either the `.bss` (uninitialised) or the `.data` (initialised) section
 - Data in `.bss` will always be **zero** at program startup
- Dynamic memory must be allocated via the system (`malloc/sbrk`)
 - Seldomly used in assembly

```
1 .bss
2     NR_BSS: .long 0      // Will be 0 at program startup
3     MORE: .space 128    // 128 bytes of zeroes
4
5 .data
6     NR_DATA: .long 0x123 // Hex value
7     .asciz "hej"        // string (i.e. bytes 0x6865600)
8
9 .section .text
10    movl NR_DATA, %eax   // NR DATA into %eax
11    movl %eax, NR_BSS    // Copy from NR_DATA to NR_BSS
```


ASM Inst. | Handling Data

- Data can be in either the `.bss` (uninitialised) or the `.data` (initialised) section
 - Data in `.bss` will always be **zero** at program startup
- Dynamic memory must be allocated via the system (`malloc/sbrk`)
 - Seldomly used in assembly

```
1 .bss
2     NR_BSS: .long 0      // Will be 0 at program startup
3     MORE: .space 128    // 128 bytes of zeroes
4
5 .data
6     NR_DATA: .long 0x123 // Hex value
7     .asciz "hej"        // string (i.e. bytes 0x6865600)
8
9 .section .text
10    movl NR_DATA, %eax    // NR_DATA into %eax
11    movl %eax, NR_BSS    // Copy from NR_DATA to NR_BSS
```

ASM Inst. | Constants

- Depends on the compiler
- With *GCC* the C preprocessor runs, so you can use *#define*.
- With “*as*”, constants are set using the *.set* keyword
 - Trick: Run *cpp* before *as* if you want to use *#define*
- *NASM* has its own preprocessor (Note the intel syntax!)

```
1 #define CONSTANT 16
2 ... // GCC
3     movl $CONSTANT, %eax
4
5 .set CONSTANT, 16
6 ... // GNU as
7     movl $CONSTANT, %eax
8
9 %define CONSTANT 16
10 ... // NASM
11 mov eax, CONSTANT
```

ASM Inst. | Constants

- Depends on the compiler
- With **GCC** the C preprocessor runs, so you can use `#define`.
- With “as”, constants are set using the `.set` keyword
 - Trick: Run `cpp` before `as` if you want to use `#define`
- **NASM** has its own preprocessor (Note the intel syntax!)

```
1 #define CONSTANT 16
2 ... // GCC
3     movl $CONSTANT, %eax
4
5 .set CONSTANT, 16
6 ... // GNU as
7     movl $CONSTANT, %eax
8
9 %define CONSTANT 16
10 ... // NASM
11 mov eax, CONSTANT
```

ASM Inst. | Constants

- Depends on the compiler
- With *GCC* the C preprocessor runs, so you can use *#define*.
- With *“as”*, constants are set using the *.set* keyword
 - Trick: Run *cpp* before *as* if you want to use *#define*
- *NASM* has its own preprocessor (Note the intel syntax!)

```
1 #define CONSTANT 16
2 ... // GCC
3     movl $CONSTANT, %eax
4
5 .set CONSTANT, 16
6 ... // GNU as
7     movl $CONSTANT, %eax
8
9 %define CONSTANT 16
10 ... // NASM
11 mov eax, CONSTANT
```

ASM Inst. | Constants

- Depends on the compiler
- With *GCC* the C preprocessor runs, so you can use *#define*.
- With “*as*”, constants are set using the *.set* keyword
 - Trick: Run *cpp* before *as* if you want to use *#define*
- **NASM** has its own preprocessor (Note the intel syntax!)

```
1 #define CONSTANT 16
2 ... // GCC
3     movl $CONSTANT, %eax
4
5 .set CONSTANT, 16
6 ... // GNU as
7     movl $CONSTANT, %eax
8
9 %define CONSTANT 16
10 ... // NASM
11 mov eax, CONSTANT
```

ASM Inst. | Instruction Format

- Instruction format: **instr [op1, op2]**
- Instructions have 0, 1 or 2 operands
 - In AT&T syntax, **instr src, dst**, and in Intel syntax **instr dst, src**
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

```
1 .data
2     label: .long 0
3 .text
4     movl $5, %rax           // register %rax = 5
5     neg %rax                // %rax = -%rax
6     movl $5, label          // put 5 in the memory at label
7     movl $label, %rax       // place the address of label in %rax
8     push (%rax)             // Push the value on label on the stack
```

ASM Inst. | Instruction Format

- Instruction format: `instr [op1, op2]`
- Instructions have 0, 1 or 2 operands
 - In AT&T syntax, `instr src, dst`, and in Intel syntax `instr dst, src`
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

```
1 .data
2     label: .long 0
3 .text
4     movl $5, %rax           // register %rax = 5
5     neg %rax                // %rax = -%rax
6     movl $5, label          // put 5 in the memory at label
7     movl $label, %rax       // place the address of label in %rax
8     push (%rax)             // Push the value on label on the stack
```

ASM Inst. | Instruction Format

- Instruction format: `instr [op1, op2]`
- Instructions have 0, 1 or 2 operands
 - In AT&T syntax, `instr src, dst`, and in Intel syntax `instr dst, src`
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

```
1 .data
2     label: .long 0
3 .text
4     movl $5, %rax           // register %rax = 5
5     neg %rax                // %rax = -%rax
6     movl $5, label          // put 5 in the memory at label
7     movl $label, %rax        // place the address of label in %rax
8     push (%rax)              // Push the value on label on the stack
```


ASM Inst. | Instruction Format

- Instruction format: `instr [op1, op2]`
- Instructions have 0, 1 or 2 operands
 - In AT&T syntax, `instr src, dst`, and in Intel syntax `instr dst, src`
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

```
1 .data
2     label: .long 0
3 .text
4     movl $5, %rax           // register %rax = 5
5     neg %rax               // %rax = -%rax
6     movl $5, label         // put 5 in the memory at label
7     movl $label, %rax      // place the address of label in %rax
8     push (%rax)            // Push the value on label on the stack
```

ASM Inst. | Instruction Format

- Instruction format: `instr [op1, op2]`
- Instructions have 0, 1 or 2 operands
 - In AT&T syntax, `instr src, dst`, and in Intel syntax `instr dst, src`
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

```
1 .data
2     label: .long 0
3 .text
4     movl $5, %rax           // register %rax = 5
5     neg %rax                // %rax = -%rax
6     movl $5, label          // put 5 in the memory at label
7     movl $label, %rax        // place the address of label in %rax
8     push (%rax)              // Push the value on label on the stack
```

ASM Inst. | Instruction Format

- Instruction format: `instr [op1, op2]`
- Instructions have 0, 1 or 2 operands
 - In AT&T syntax, `instr src, dst`, and in Intel syntax `instr dst, src`
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

```
1 .data
2     label: .long 0
3 .text
4     movl $5, %rax           // register %rax = 5
5     neg %rax                // %rax = -%rax
6     movl $5, label          // put 5 in the memory at label
7     movl $label, %rax       // place the address of label in %rax
8     push (%rax)             // Push the value on label on the stack
```

ASM Inst. | Instruction Format

- Instruction format: **instr** [**op1**, **op2**]
- Instructions have 0, 1 or 2 operands
 - In *AT&T syntax*, **instr** **src**, **dst**, and in *Intel syntax* **instr** **dst**, **src**
- The operands can be (i) an immediate operand (a constant), (ii) a register, or (iii) a memory reference.

Spaces / Tabs not necessary

```
1 .data
2 label: .long 0
3 .text
4 movl $5, %rax           // register %rax = 5
5 neg %rax                // %rax = -%rax
6 movl $5, label          // put 5 in the memory at label
7 movl $label, %rax        // place the address of label in %rax
8 push (%rax)             // Push the value on label on the stack
```

ASM Inst. | Labels

- Labels point out memory addresses
- Compile-time replaced with absolute or relative addresses
- **Local labels** are numbers (labels without symbols)
 - Direction specified on reference (*f* or *b*)
- Global labels can be seen from other files (but not always)

```
1 .globl fn           // Declare fn as global
2 fn:                // The fn label
3 lab:               // This is also a label, visible in this file
4     jmp lab        // This is a jump to lab (infinite loop)
5     jmp 1f         // Jump to 1 (forward)
6 1:                 // Local label
7     jmp 1b         // Jump to 1 (backwards)
8     jmp 1f         // Jump to 1 (forward)
9 1:                 // Local label again
```

ASM Inst. | Labels

- Labels point out memory addresses
- Compile-time replaced with absolute or relative addresses
- Local labels are numbers (labels without symbols)
 - Direction specified on reference (*f* or *b*)
- Global labels can be seen from other files (but not always)

```
1 .globl fn           // Declare fn as global
2 fn:                // The fn label
3 lab:               // This is also a label, visible in this file
4     jmp lab        // This is a jump to lab (infinite loop)
5     jmp 1f         // Jump to 1 (forward)
6 1:                 // Local label
7     jmp 1b         // Jump to 1 (backwards)
8     jmp 1f         // Jump to 1 (forward)
9 1:                 // Local label again
```

ASM Inst. | Labels

- Labels point out memory addresses
- Compile-time replaced with absolute or relative addresses
- Local labels are numbers (labels without symbols)
 - Direction specified on reference (*f* or *b*)
- **Global labels can be seen from other files** (but not always)

```
1 .globl fn           // Declare fn as global
2 fn:                // The fn label
3 lab:               // This is also a label, visible in this file
4     jmp lab        // This is a jump to lab (infinite loop)
5     jmp 1f         // Jump to 1 (forward)
6 1:                 // Local label
7     jmp 1b         // Jump to 1 (backwards)
8     jmp 1f         // Jump to 1 (forward)
9 1:                 // Local label again
```

ASM Inst. | Registers

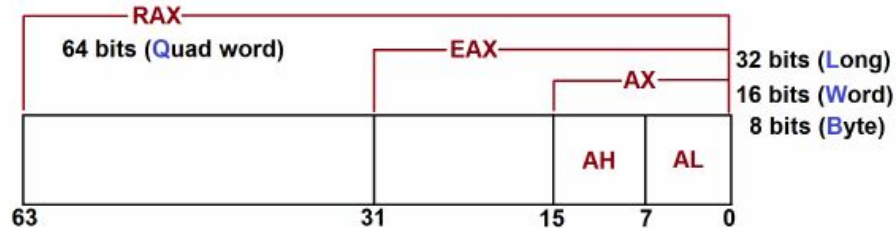
- IA-32 has 8 general-purpose registers
- The registers have special uses in some instructions
 - `%eax`: Accumulator, destination of `mul` etc
 - `%ebx`: (Sometimes pointer to data)
 - `%ecx`: Counter for loops
 - `%edx`: Additional destination register
 - `%esi`: Source for string ops
 - `%edi`: Destination for string ops
 - `%esp`: Stack pointer
 - `%ebp`: Frame pointer
- There are also some special registers:
 - `%eip`: Program counter (not directly accessible)
 - `%eflags`: Status flags, accessible through special instructions
 - Plus segmentation `cs`, `ds`, etc. Generally not usable outside the OS kernel

ASM Inst. | Registers

- **x86-64** has **16** general-purpose registers
- The registers have special uses in some instructions
 - **%rax**: Accumulator, destination of mul etc
 - **%rbx**: (Sometimes pointer to data)
 - **%rcx**: Counter for loops
 - **%rdx**: Additional destination register
 - **%rsi**: Source for string ops
 - **%rdi**: Destination for string ops
 - **%rsp**: Stack pointer
 - **%rbp**: Frame pointer
 - **%r8-%r15**: Additional registers
- There are also some special registers:
 - **%rip**: Program counter (not directly accessible)
 - **%rflags**: Status flags, accessible through special instructions
 - Plus segmentation *cs*, *ds*, etc. Generally not usable outside the OS kernel

ASM Inst. | Registers

- Some registers have directly accessible 8, 16, and 32-bit parts.
- [Yossarian's blog](#) includes a more details on all existing registers.



From 2023 lecture by Sai Prashanth Josyula

64-bit	32-bit	16-bit	8-bit (low)
RAX	EAX	AX	AL
RBX	EBX	BX	BL
RCX	ECX	CX	CL
RDX	EDX	DX	DL
RSI	ESI	SI	SIL
RDI	EDI	DI	DIL
RBP	EBP	BP	BPL
RSP	ESP	SP	SPL
R8	R8D	R8W	R8B
R9	R9D	R9W	R9B
R10	R10D	R10W	R10B
R11	R11D	R11W	R11B
R12	R12D	R12W	R12B
R13	R13D	R13W	R13B
R14	R14D	R14W	R14B
R15	R15D	R15W	R15B

From 2020 blog post by yossarian

Outline

- Overview
 - Lab, Supervision, and Assignment
- Motivation
 - Why use Assembly?
 - IA-32 & x86-64 History
 - IA-32 & x86-64 Architecture
- Assembly Language Instruction
 - Compiling and Writing Assembly Language
 - Handling of Data (variables & constants)
 - Instruction Format, Labels, and Registers
- Writing IA-32 / x86-64 Programs
 - Arithmetic / Logic Instructions
 - Program Flow (branches)
 - Referencing Memory
 - Interfacing with the Operating System

ASM Inst. | Arithmetic / Logic Instructions

- Generally: ***opsize src, dst***
 - Except *mul* and *div*
- Keep this slide under your pillow

```
1 addl %eax, %ebx    // %ebx = %ebx + %eax
2 subl $0xff, %ebx   // %ebx = %ebx - 255
3 andl $3, %eax      // %eax = %eax & 3, i.e. zero bits 2..31
4 xorl %eax, %eax     // %eax = %eax ^ %eax (i.e. zero %eax)
5 orl $4, %ecx       // %ecx = %ecx | 4, i.e. set bit 2
6 shll $2, %eax      // %eax = %eax << 2, i.e. %eax = %eax * 4
7 subb $8, %ah       // %ah = %ah - 8 (sub 8 from %ah, byte-size)
8 addl (%eax), %ecx   // %ecx = %ecx + (%eax)
9 subw $8, 8         // (8) = (8) - 8 (i.e. the address 8!, word-size)
10 mull %ecx          // %edx:%eax = %eax * %ecx, %edx = bits 32..63
11 divl %ebx          // %eax = %edx:%eax / %ebx, %edx = %edx:%eax MOD %ebx
12 mull (%ebx)        // %edx:%eax = (%ebx) * %eax (memory references)
```

ASM Inst. | Program Flow (unconditional jumps)

- Unconditional jumps are done with the `jmp` instruction
- The target can be either of:
 - An absolute address (a 64 bit value)
 - A relative address (± 128 bytes from the current `%rip`)
 - A relative address ($\pm 4G$ bytes from the current `%rip`)
 - A register (`jmp *%rax`), not necessary `%rax`, *any register works*
- The compiler determines if a relative jump can be done and chooses
- the smallest encoding

```
1  jmp label
2  ...
3  movq $label, %rax    // The address of label in %eax
4  jmp *%rax            // Jump to the contents of %eax
5  label:
```

ASM Inst. | Program Flow (conditional jumps)

- Conditional jumps are done with the *jcc* label instructions where *cc* is a condition
 - The conditions come from flags in the *%RFLAGS* register
 - The test and cmp instructions can be used to set *RFLAGS* before a conditional jump
- There are many of these. Examples:
 - *jz*: Jump if zero
 - *jnz*: Jump if not zero
 - *je*: Jump if equal, same as *jz* (jump if zero-flag set)
 - *jne*: Jump if not equal
 - *jl*: Jump if less than
 - *jg*: Jump if greater than
 - *jle*: Jump if less than or equal
 - *jge*: Jump if greater than or equal
- The target of a conditional jump is **always** a label

ASM Inst. | Program Flow (conditional jumps), if / else

- If-else statements can be constructed with conditional jumps:
- *cmpq* performs a “mental” subtraction between the arguments and sets the flags (so a jump can be done)
- In the example, *%rax-1* is done, and the zero-flag is set if *%rax* is 1
 - *test* performs a “mental” bitwise instead

```
1      cmpq $1, %rax    // if (%rax = 1)
2      je   then        //      %rbx = 3
3      movq $2, %rbx    // else
4      jmp  out         //      %rbx = 2
5 then:  movq $3, %rbx
6 out:
```

ASM Inst. | Program Flow (conditional jumps), if / else

- If-else statements can be constructed with conditional jumps:
- *cmpq* performs a “mental” subtraction between the arguments and sets the flags (so a jump can be done)
- In the example, *%rax-1* is done, and the zero-flag is set if *%rax* is 1
 - *test* performs a “mental” bitwise instead
- How can we avoid the *jmp* out instruction in the false-branch?

```
1      cmpq $1, %rax    // if (%rax = 1)
2      je   then        //      %rbx = 3
3      movq $2, %rbx    // else
4      jmp  out          //      %rbx = 2
5 then:  movq $3, %rbx
6 out:
```


ASM Inst. | Program Flow (conditional jumps), if / else

- If-else statements can be constructed with conditional jumps:
- *cmpq* performs a “mental” subtraction between the arguments and sets the flags (so a jump can be done)
- In the example, *%rax-1* is done, and the zero-flag is set if *%rax* is 1
 - *test* performs a “mental” bitwise instead
- How can we avoid the *jmp* out instruction in the false-branch?

```
1      cmpq $1, %rax
2      je   then
3      movq $2, %rbx
4      jmp  out
5 then:  movq $3, %rbx
6 out:
```

```
1      movq $3, %rbx    // %rbx = 3
2      cmpq $1, %rax    // if !(%rax = 1)
3      je   then        //      %rbx = 2
4      movq $2, %rbx
5 then:
```

ASM Inst. | Program Flow (conditional jumps), loops

- Loops can be constructed with jumps
- The example performs *for(i=0; i<5; i++);*
- Substitute the ... for your loop body
- If you can loop “backwards” (i.e. starting at 4 in this example), there is a better way!

```
1  xorq %rax, %rax // zero %rax
2  l1: cmpq $5, %rax // if (%rax == 5)
3      je out      // exit loop
4      ...         // Do something in the loop here!
5      incq %rax   // %rax++
6      jmp l1      // loop
7  out:
```

ASM Inst. | Program Flow (conditional jumps), loops

- You can also use the *loop* instruction for loops
- *loop* decrements *%rcx* with 1 and jumps if *%rcx* does not become 0
- There are also *loopcc*, where loops are conditioned.
 - The example loops if *%rbx* $\neq 0$ and *%rcx* does not become 0

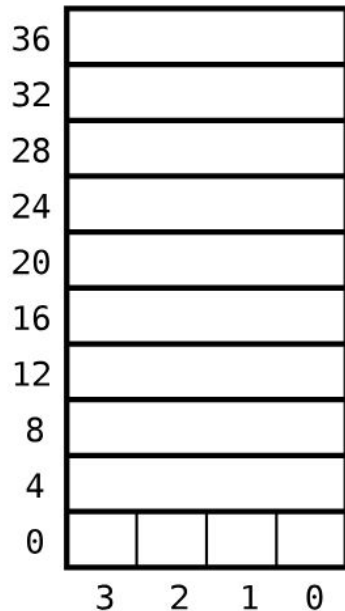
```
1    movq $4, %rcx    // Init %rcx
2  1:    ...          // Do something here!
3    loop 1b          // %rcx--; if (%rcx  $\neq$  0) goto 1b
4  1:    ...
5    cmpq $0, %rbx    // set %rflags according to %rbx-0
6    loopne 1b        // loop
```

ASM Inst. | Program Flow (conditional jumps), loops

- You can also use the *loop* instruction for loops
- *loop* decrements *%rcx* with 1 and jumps if *%rcx* does not become 0
- There are also *loopcc*, where loops are conditioned.
 - The example loops if *%rbx* $\neq 0$ and *%rcx* does not become 0

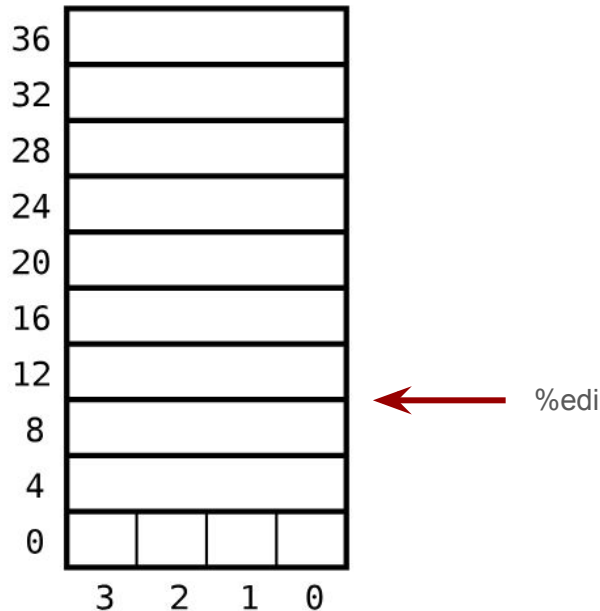
```
1  movq $4, %rcx    // Init %rcx
2  1:  ...          // Do something here!
3  loop 1b          // %rcx--; if (%rcx  $\neq$  0) goto 1b
4  1:  ...
5  cmpq $0, %rbx    // set %rflags according to %rbx-0
6  loopne 1b        // loop
```

ASM Inst. | Memory References



- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $\text{address} = \text{offset}(\text{base reg}, \text{index reg}, \text{scale})$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`

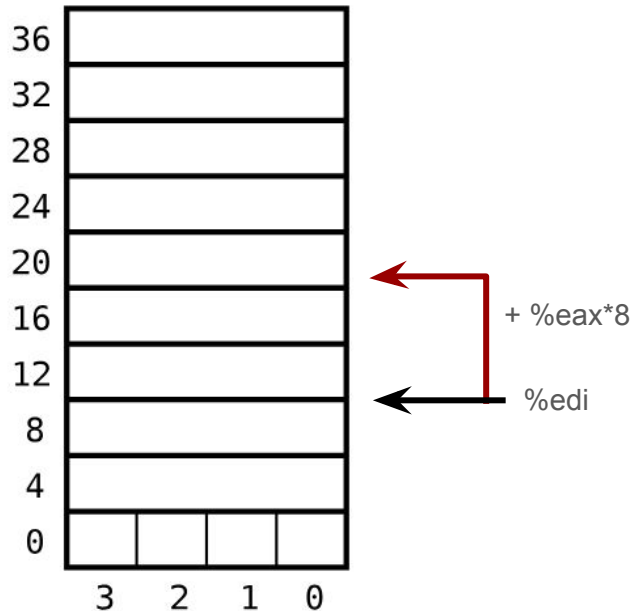
ASM Inst. | Memory References



- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $\text{address} = \text{offset}(\text{base reg}, \text{index reg}, \text{scale})$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`

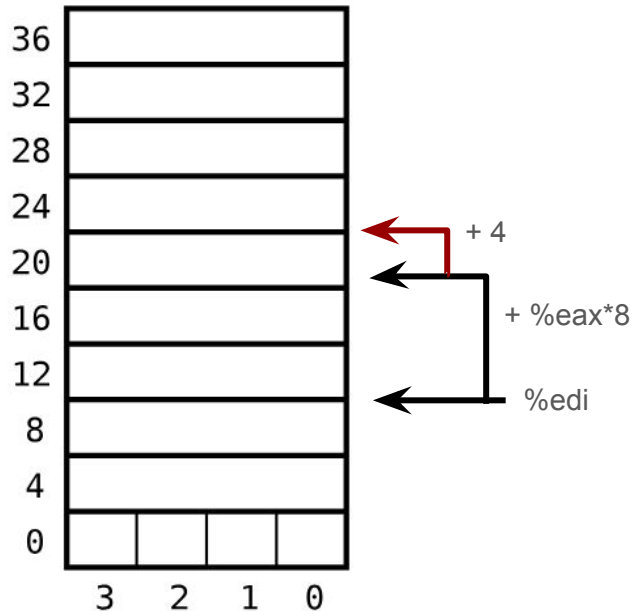
ASM Inst. | Memory References

- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $\text{address} = \text{offset}(\text{base reg}, \text{index reg}, \text{scale})$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`



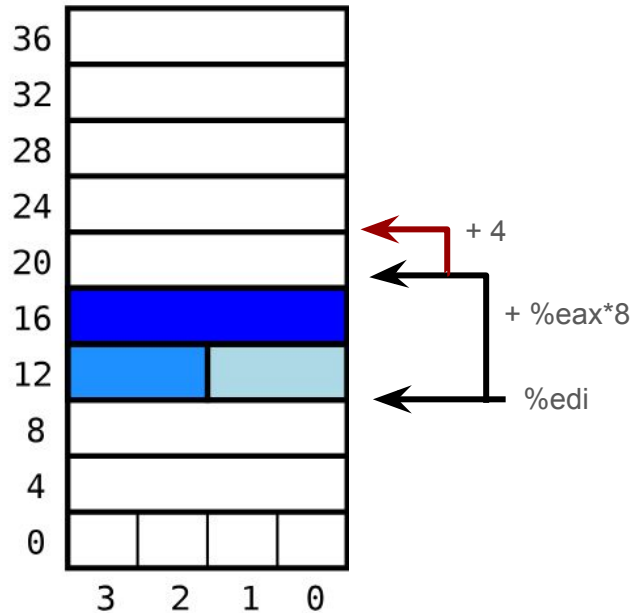
ASM Inst. | Memory References

- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $\text{address} = \text{offset}(\text{base reg}, \text{index reg}, \text{scale})$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`



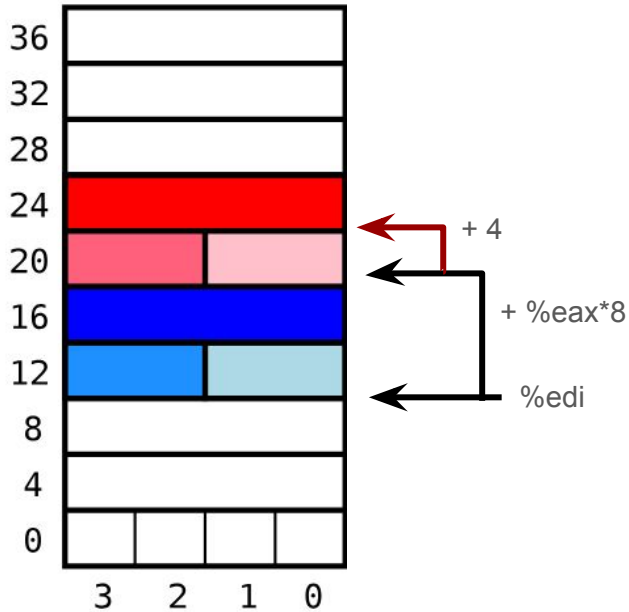
ASM Inst. | Memory References

- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $address = offset(base\ reg, index\ reg, scale)$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`
- Useful for handling data structures



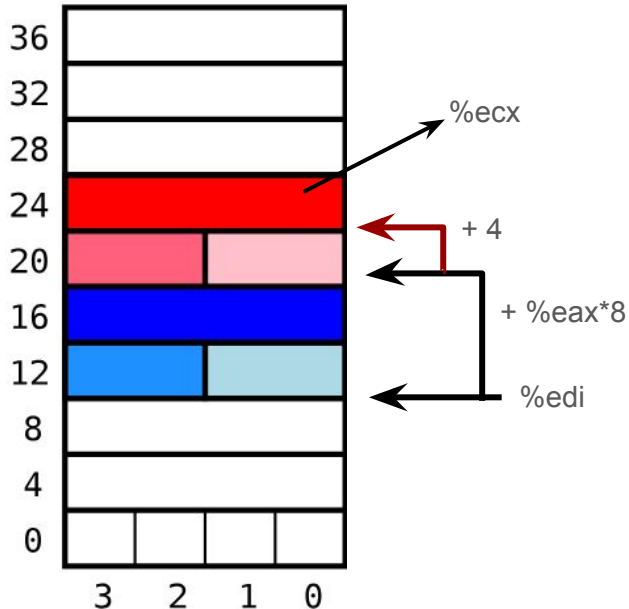
ASM Inst. | Memory References

- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $address = offset(base\ reg, index\ reg, scale)$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`
- Useful for handling data structures



ASM Inst. | Memory References

- Most x86/x86-64 instructions can do memory references
- Memory references are quite complex:
 - $address = offset(base\ reg, index\ reg, scale)$
- Example:
 - `movl 4(%edi, %eax, 8), %ecx`
- Useful for handling data structures
- There are shorter forms:
 - `addb $1, (%edi)`
 - only base register
 - `movl 4(%esp), %eax`
 - base and offset



ASM Inst. | Memory References, more

- You can only have one memory reference per instruction
- Memory references can also be absolute (1,2,4)
- but beware: (2 and 3) looks similar, but
 - (1,2, and 4) copies the value from an address
 - (3 and 5) copies the address

```
1 movl 0x1000, %eax    // Copy 4 bytes from address 4096
2 movl label, %eax     // Copy 4 bytes from address 'label'
3 movl $label, %eax    // Copy the value label into %eax
4 movl 128, %eax       // Copy 4 bytes from address 128
5 movl $128, %eax      // Copy the value 128 into %eax
```

ASM Inst. | Memory References, more

- You can only have one memory reference per instruction
- Memory references can also be absolute (1,2,4)
- but beware: (2 and 3) looks similar, but
 - (1,2, and 4) copies the value from an address
 - (3 and 5) copies the address

```
1 movl 0x1000, %eax // Copy 4 bytes from address 4096
2 movl label, %eax  // Copy 4 bytes from address 'label'
3 movl $label, %eax // Copy the value label into %eax
4 movl 128, %eax    // Copy 4 bytes from address 128
5 movl $128, %eax   // Copy the value 128 into %eax
```

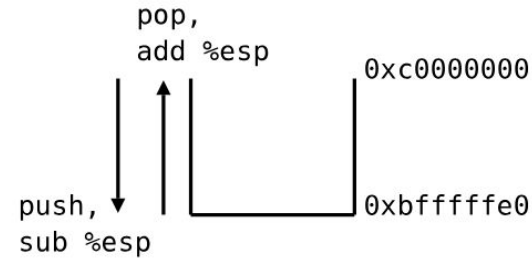
ASM Inst. | Memory References, more

- You can only have one memory reference per instruction
- Memory references can also be absolute (1,2,4)
- but beware: (2 and 3) looks similar, but
 - (1,2, and 4) copies the value from an address
 - (3 and 5) copies the address

```
1 movl 0x1000, %eax // Copy 4 bytes from address 4096
2 movl label, %eax // Copy 4 bytes from address 'label'
3 movl $label, %eax // Copy the value label into %eax
4 movl 128, %eax // Copy 4 bytes from address 128
5 movl $128, %eax // Copy the value 128 into %eax
```

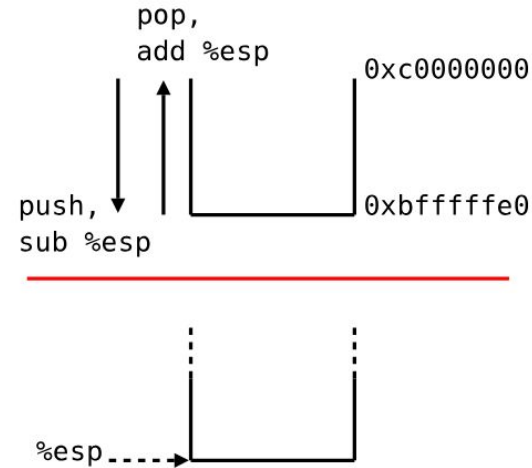
ASM Inst. | The Stack

- The stack is where “automatic” (local) variables are stored
- The stack top is stored in `%rsp`
 - `addq $8, %rsp` **decreases** the stack size with 8 bytes
 - `subq $8, %rsp` **increases** the stack size with 8 bytes
- `push` pushes a value on the stack
- `pop` pops a value from the stack



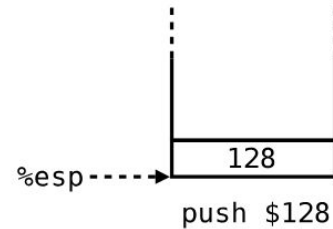
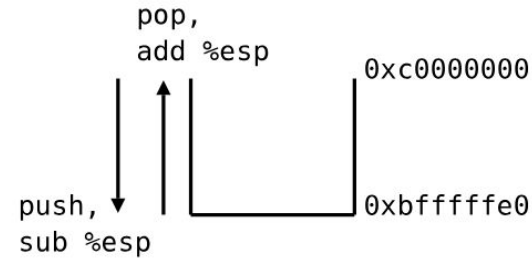
ASM Inst. | The Stack

- The stack is where “automatic” (local) variables are stored
- The stack top is stored in `%rsp`
 - `addq $8, %rsp` **decreases** the stack size with 8 bytes
 - `subq $8, %rsp` **increases** the stack size with 8 bytes
- `push` pushes a value on the stack
- `pop` pops a value from the stack



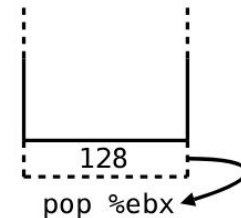
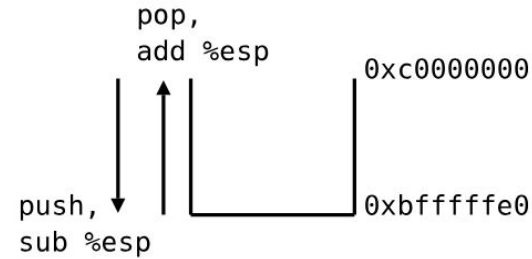
ASM Inst. | The Stack

- The stack is where “automatic” (local) variables are stored
- The stack top is stored in `%rsp`
 - `addq $8, %rsp` **decreases** the stack size with 8 bytes
 - `subq $8, %rsp` **increases** the stack size with 8 bytes
- **push pushes a value on the stack**
- `pop` pops a value from the stack

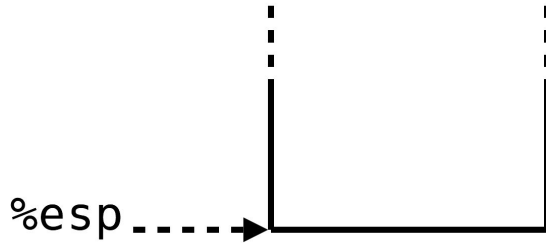


ASM Inst. | The Stack

- The stack is where “automatic” (local) variables are stored
- The stack top is stored in `%rsp`
 - `addq $8, %rsp` **decreases** the stack size with 8 bytes
 - `subq $8, %rsp` **increases** the stack size with 8 bytes
- `push` pushes a value on the stack
- `pop` pops a value from the stack



ASM Inst. | Function Calls (C calling convention) I

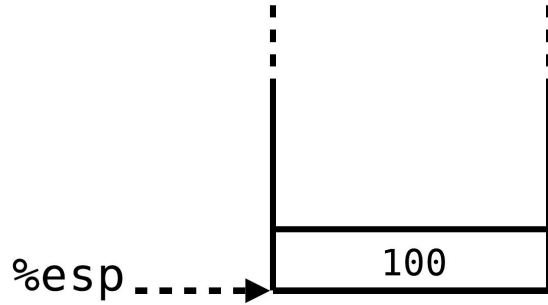


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- `%eax` contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) I

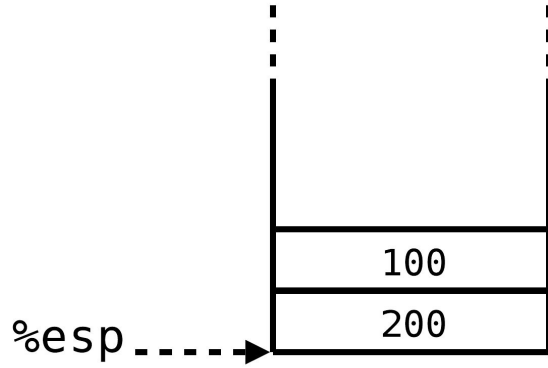


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- `%eax` contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) I

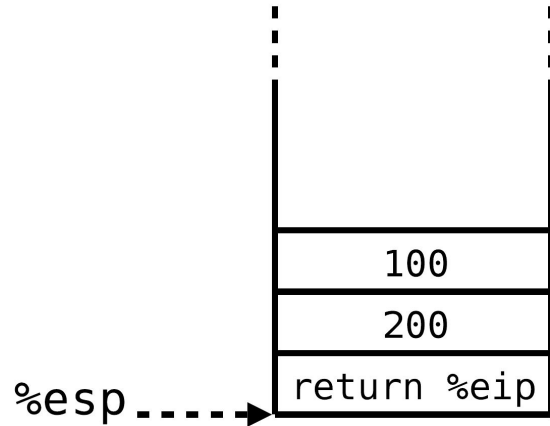


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- *%eax* contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) I

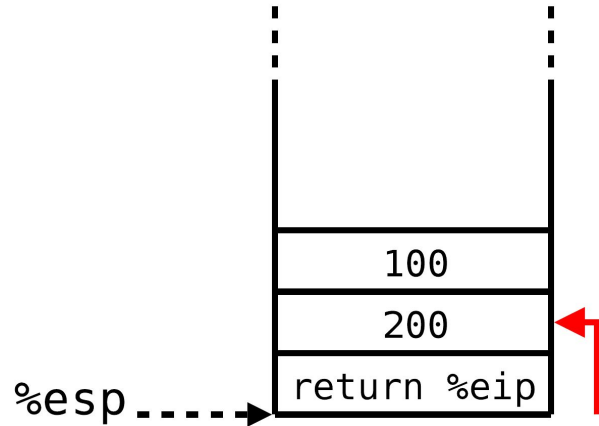


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- *%eax* contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) I

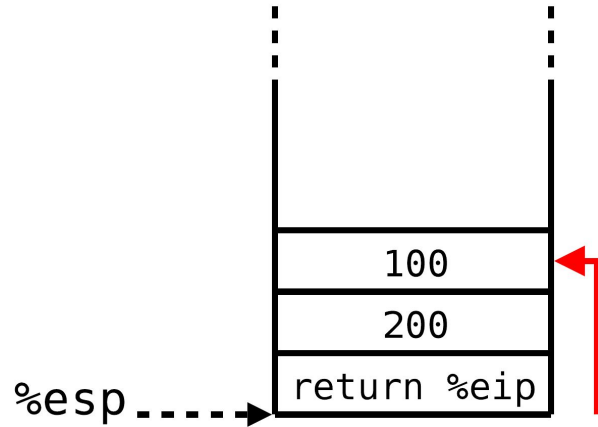


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- *%eax* contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) I

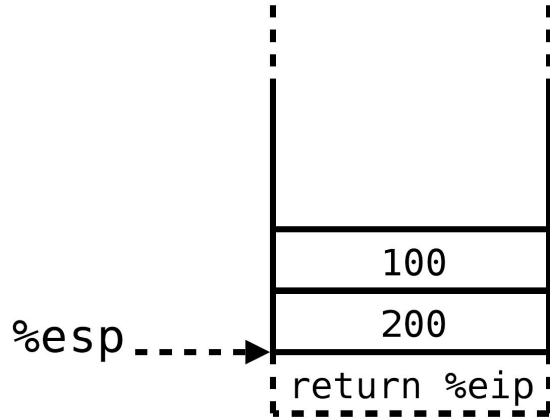


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- *%eax* contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```


ASM Inst. | Function Calls (C calling convention) I

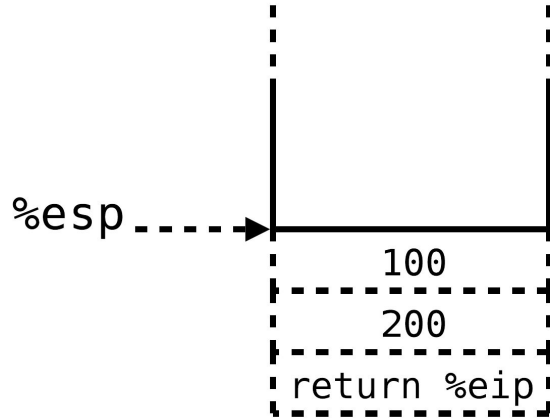


- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- *%eax* contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) I



- Arguments are pushed in reverse order on the stack
- The *PC* (program counter) is pushed on *call* and popped on *ret*
- `%eax` contains the return value
- The stack pointer is restored by the caller

```
1 push $100
2 push $200
3 call fn
4 addl $8, %esp
```

```
1 fn: movl 4(%esp), %eax
2     movl 8(%esp), %edx
3     ...
4     movl $0, %eax
5     ret
```

ASM Inst. | Function Calls (C calling convention) II

- Arguments are always passed on the stack in reverse order.
- `%rsp`, `%rbp`, `%rbx`, `%rsi`, `%rdi` must not be changed by the function
- `%rax` contains the result of the function.
- Passed arguments may be overwritten by the function
- The caller must restore the stack (i.e. pop the arguments)
- Arguments should foremost be passed via registers.
 - Arguments are passed via `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`, then on the stack.
 - Which registers used is architecture dependent.
 - See “syscall” man page for which register to use when calling a system call. They differ slightly compared to user-level functions!

ASM Inst. | Basic UNIX program I

- A very simple UNIX program is the following (simple.asm)
- Compile with the “as”¹ command: `as -gstabs simple.asm -o simple.o`
 - `-gstabs`: Supply debugging information for GDB.
 - Other arguments: See “as” man page
- Link with the “gcc”² command: `gcc -nostdlib -lc -o simple simple.o`
 - `-nostdlib`: Only link to libraries we specify in the command.
 - `-lc`: Link to `libc`
 - “`ld`” can also be used³
- Programs linked with “`ld`” or “`gcc`” always start (entrypoint) at the `_start` label.
 - Remember to add the entrypoint to the object code’s export table (`.global` or `.globl` directive), so that the linker can find it.
 - You can change the entrypoint label with the “`-e <label>`” or “`--entry <label>`” flag specific to the linker.

```
1 .globl _start      /* Required label for the entry point */
2 .section .text
3 _start:
4     movq $2, %rdi    /* Specify the exit code */
5     call exit        /* Call the exit function */
```

1: “as” documentation is linked in the [GNU's binutils webpage](https://sourceware.org/binutils/docs/as.html), the documentation itself is at <https://sourceware.org/binutils/docs/as.html>

2: “gcc” documentation is linked in the [GNU's GCC webpage](https://gcc.gnu.org/onlinedocs/), the documentation itself is at <https://gcc.gnu.org/onlinedocs/>

3: “ld” documentation is linked in the [GNU's binutils webpage](https://sourceware.org/binutils/docs/ld.html), the documentation itself is at <https://sourceware.org/binutils/docs/ld.html>

ASM Inst. | Basic UNIX program II

- The **exit** system call which we used in the last program was called *indirectly* via **libc**
- To call it directly, we must resort to system dependent methods
 - *syscall* (x86-64) or *int \$0x80* (x86, but prefer *sysenter*)
- System calls on Linux differ greatly depending on architecture
 - see “syscall” man page (specifically section **Architecture calling conventions**)

```
1 .globl _start    /* Required label for the entry point */
2 .section .text
3 _start:
4     movq $60, %rax /* Specify the system call for exit */
5     movq $2, %rdi  /* Specify the exit code */
6     syscall        /* Make the system call */
```

ASM Inst. | Example: Factorial calculation

- Factorial example: $5! = 5 * 4 * 3 * 2 * 1$

```
1 fact:
2     mov 4(%rsp), %rcx    /* Load the argument into %rcx */
3     mov $1, %rax        /* Init result */
4 1:
5     mul %rcx            /* Multiply %rax by %rcx */
6     loop 1b            /* Decrement %rcx and loop if not 0 */
7     ret
8
9     ...
10
11     push $5            /* Push the argument */
12     call fact
```

ASM Inst. | Example: Adder

- Add all numbers in the a list of numbers (*arr*)
- Stop loop when zero is encountered

```
1  .data
2      arr:                /* array of 64-bit integers */
3          .quad 1
4          .quad 2
5          .quad 3
6          .quad 0
7  .text
8  adder:
9      xorq %rax, %rax      /* zero %rax */
10     movq $arr, %rbx      /* load address of arr into %rbx */
11  inner:
12     movq (%rbx), %rcx     /* load next element of arr into %rcx */
13     cmpq $0, %rcx        /* if %rcx = 0 */
14     je out
15     addq %rcx, %rax       /* (else) %rax += %rcx */
16     addq $8, %rbx        /* increment %rbx */
17     jmp inner
18  out:
19     ret
```

ASM Inst. | Example: Frame/Base pointer

- C code, with corresponding assembly code to the right

```
1 void proc(int a) { int b = a; }
```



Set up the stack frame, i.e.

```
1 pushq %rbp      /* Save old %rbp */
2 movq %rsp, %rbp /* Set %rbp to point to current %rsp */
```

- The above code is identical to an instruction called “enter”, but its unused by GCC as it causes performance issues.

“leave” is an instruction that destroys a stack frame, i.e.

```
1 movq %rbp, %rsp /* Set %rsp to point to old %rbp */
2 pop %rbp        /* Restore old %rbp */
```

```
1 proc:
2     pushq %rbp
3     movq %rsp, %rbp
4     # Allocate space for one local variable
5     subq $8, %rsp
6     # Get the argument "a"
7     # Remember that we must jump over the return address
8     # to get to arguments on the stack (hence the 16)
9     # Stack layout:
10    # "a" ← 16(%rbp)
11    # %rip ← 8(%rbp)
12    # %rbp ← 0(%rbp)
13    movq 16(%rbp), %rax # First argument "a"
14    movq %rax, -8(%rbp) # Save "a" into local variable "b"
15    leave              /* Clean up stack frame */
16    ret
17
18    ...
19
20    pushq $42
21    call proc
```